WHITE LANE AI

Unified Command Center Plan

Strategic Analysis & Technical Review

Prepared by Jessica Mitchell & Philo (Workshop Team)

Date April 8, 2026

Classification Internal — For Argos Review

Subject Analysis of command-center-plan.html (v1.0)

Executive Summary

The Unified Command Center Plan proposes merging three parallel UI streams — Isaac's Mission Control (Next.js/Vercel), Argos's SSDA screens (vanilla HTML), and Jess's Command Center navigation — into a single iframe-based shell application.

This analysis validates the plan's technical claims against the actual codebase, evaluates architectural tradeoffs against industry standards, identifies risks and gaps, and provides strategic recommendations.

Our assessment: The plan is architecturally sound for the current stage. The iframe shell is the correct Phase 1 approach — it ships in days, not weeks, and respects team autonomy. However, the plan contains several factual inaccuracies about current screen state, underestimates cross-origin complexity, and needs a clearer migration path to a component-based architecture for Phase 5.

1. Plan Accuracy — Verified Against Codebase

We verified every claim in the plan against the live deployed screens on `ssda.whitelanewrx.com` and the actual source code in the WhiteLaneWrx repository.

1.1 Screen Inventory Verification

The plan claims 16 menu items with 12 built and 4 planned. Our audit of the actual `app/wrx-005/screens/` directory found **16 HTML files** plus Isaac's Vercel app. Here's what we verified:

SCREEN	PLAN STATUS	ACTUAL STATUS	NOTES
Command Center	BUILT (MERGE)	BUILT	`command-center-live.html` — has its own sidebar nav. Does NOT include `project-picker.js`.
Plan	BUILT (ISAAC)	BUILT on Vercel	Floor plan + room explorer. Vercel deployment confirmed.
Scope	BUILT (MERGE)	BUILT	`scope-explorer-live.html` + `scope-mst.html` — two separate screens, not one.
Schedule	BUILT (ISAAC)	BUILT on Vercel	Gantt by trade. Also exists as `gantt.html` in services/mission-control.
Field Activity	BUILT (ISAAC)	BUILT on Vercel	Photo/flag/note feed from Kestrel.
Documents	BUILT (MERGE)	BUILT	`document-portal.html` + `file-explorer.html` — two separate screens.
Estimate Editor	BUILT (ARGOS)	BUILT	Fully functional. S/G/P tiers, rooms, markup, proposal print.
Contract Control	BUILT (ARGOS)	BUILT	Checkpoints, lifecycle, diff viewer.
Money	BUILT (ARGOS)	BUILT	`project-manifest.html` — tab-based manifest with 7 sections.
Change Orders	PLANNED (JESS)	NOT BUILT	No file exists.
Selections	BUILT (JESS)	BUILT	10-state lifecycle, dual-view. Has its own sidebar.
Subs & Vendors	PLANNED (JESS)	NOT BUILT	No file exists.
PM Inbox	PLANNED (JESS)	NOT BUILT	`pm-inbox.html` exists in parent dir but not in screens/.
Daily Briefing	BUILT (ARGOS)	BUILT	Newspaper layout. Fully functional.
Price Book	BUILT (ARGOS)	BUILT	Product CRUD, tier editing, kits.
Analytics	PLANNED (JESS)	NOT BUILT	No file exists.

Finding: Screen count is accurate (12 built, 4 planned). However, several "single" menu items map to 2 separate HTML files (Scope, Documents), which the plan doesn't address.

1.2 Project Picker & Theme Support

The plan states: *"All screens support `[data-theme='dark']` via shared CSS from `project-picker.js`."*

Actual state:

- **12 of 16 screens** include `project-picker.js`
- `command-center-live.html` **does NOT** — it has its own complete sidebar navigation with 10 menu items
- **Isaac's Mission Control (Vercel)** does not use `project-picker.js` — it's a Next.js app with its own styling

Implication: The claim that "all screens" support the shared theme system is incorrect. The most important screen — the Command Center itself — operates independently.

1.3 iframe Detection

The plan states (Phase 2): *"Hide project picker bar when inside iframe."*

Actual state: Zero screens currently contain `window.parent !== window` detection. This code has not been written. The plan's language in Section 4 reads as if this already works, but it does not.

1.4 Cross-Origin iframe Feasibility

We tested the actual HTTP headers on both deployments:

DEPLOYMENT	X-FRAME-OPTIONS	CORS	EMBEDDABLE?
Isaac's Vercel	**Not set**	`access-control-allow-origin: *`	YES
SSDA screens	**Not set**	Not set (same-origin)	YES

Good news: Neither deployment blocks iframe embedding today. However, this permissive configuration may change if Vercel adds default security headers or if Isaac adds authentication.

2. Architecture Analysis

2.1 Why iframe Shell Is Correct for Phase 1

Team independence: Three developers (Isaac, Argos, Jess) working on three different tech stacks (Next.js, vanilla HTML, vanilla HTML) need to ship without blocking each other. Iframes provide complete isolation — a bug in Isaac's React code can't crash Argos's estimate editor.

Speed to market: An iframe shell can be built and deployed in 1-2 days. Rewriting 12+ screens into a shared component framework would take weeks to months.

Zero migration cost: Every existing screen works as-is. No code rewrites, no dependency changes, no build pipeline changes.

Industry validation: Martin Fowler's canonical micro-frontends article identifies iframes as a legitimate composition technique when "hard isolation" is the priority. For a startup with 3 parallel dev streams, this is the right constraint to optimize for.

2.2 What the Industry Says

We researched current (2025-2026) micro-frontend architecture patterns:

APPROACH	BEST FOR	DRAWBACK	BUILDERWRX FIT
iframes	Hard isolation, mixed stacks, rapid deploy	Performance overhead, no shared state	BEST for Phase 1
Module Federation	Shared runtime, single framework	Next.js moving away from it; no vanilla HTML	NOT viable now
Web Components	Cross-framework reuse	No SSR (except Stencil), learning curve	FUTURE option
single-spa / qiankun	Multi-framework migration	Heavy orchestration layer	OVERKILL
Next.js Multi-Zones	All-Next.js teams	Requires all apps to be Next.js	NOT viable

Conclusion: iframes are the only approach that works for BuilderWrx's mixed tech stack without requiring rewrites. The plan made the right call.

2.3 Competitive Context — How Procore Does It

Procore — the industry's $10B+ construction management platform — uses a unified platform architecture with modular screens organized into product lines (Preconstruction, Project Management, Financial Management, Workforce). BuilderWrx's proposed sidebar groups (Operations, Estimate & Contract, People & Selections, Intelligence) mirror this pattern at a fraction of the cost.

Strategic signal: Procore is investing in AI agents (announced Q4 2024) with an "Agent Studio" for custom agents. BuilderWrx's Athena/Iris intelligence layer is architecturally ahead — Procore's AI is still limited to generative text for client updates.

3. Risk Assessment

3.1 Critical Risks

HIGH RISK

Risk 1: Cross-Origin postMessage Theme Propagation

Isaac's Next.js app uses hardcoded dark theme (`bg-[#1B2332]`). It does NOT listen for `postMessage` events. Argos's screens use `project-picker.js` which reads `localStorage` — but cross-origin iframes have separate localStorage.

> **Recommendation:** For Phase 1, limit theme toggle to Argos's screens only. Isaac's app can keep its dark theme. Add cross-origin theme sync in Phase 4 when Isaac integrates.

HIGH RISK — FUTURE

Risk 4: Authentication Architecture

Same-origin cookies are the right auth pattern, but ONLY if all apps are on the same domain. Isaac's app is on `vercel.app` — different domain, no cookie sharing.

> **Recommendation:** Deploy Isaac's app behind nginx on the same domain (`ssda.whitelanewrx.com/mc/`). This eliminates all cross-origin issues — cookies, localStorage, postMessage, theme — in one architectural move. This should be the Phase 4 target.

3.2 Moderate Risks

MEDIUM RISK

Risk 2: Project Selector State Desynchronization

If a user changes the project inside a screen's project-picker bar, the shell sidebar won't know. The two selectors will show different projects.

> **Recommendation:** Implement a two-way `postMessage` contract:
> Shell → iframe: `{ type: 'SET_PROJECT', projectId: '...' }`
> iframe → shell: `{ type: 'PROJECT_CHANGED', projectId: '...' }`

MEDIUM RISK

Risk 3: Navigation State & Browser History

If a user clicks through 5 screens in the sidebar then presses Back, they'll navigate backward within the last iframe — not back to the previous sidebar screen.

> **Recommendation:** The shell should push to `window.history` with state objects on every sidebar click and handle `popstate`. This is a standard iframe shell pattern and should be **Phase 1**, not deferred.

MEDIUM RISK

Risk 6: Screens with Built-In Sidebars (Double Sidebar)

`command-center-live.html` and `selection-tracker-live.html` both have their own sidebar navigation. When loaded inside the shell, users will see two sidebars — creating visual confusion and wasting horizontal space.

> **Recommendation:** Add an `?embedded=true` parameter that hides the internal sidebar. This should be **Phase 1**.

3.3 Low Risks

LOW RISK

Risks 5, 7, 8, 9: Performance, Print, Keyboard, Mobile

Performance: Only one iframe active at a time — acceptable.
Print: Add "Open in new tab" button to shell header.
Keyboard: Shell captures events only when sidebar is focused.
Mobile: Defer to Phase 3. Field users use Kestrel on mobile, not Command Center.

4. Strategic Assessment

4.1 Ownership Model — What It Means for Jessica

The plan assigns Jessica four screens plus the shell sidebar. Estimated total: 6-8 days of focused work.

PRIORITY	SCREEN	TIMELINE	RATIONALE
1	**Shell Sidebar**	Day 1-2	Container everything else lives in
2	**PM Inbox**	Day 3-4	Highest operational value — PMs need a decision queue
3	**Change Orders**	Day 5-6	Critical for Keiper (T&M with mental fixed pricing)
4	**Subs & Vendors**	Week 2	COI tracking, less urgent
5	**Analytics**	Week 2+	Start with basic charts, iterate

4.2 What's Missing from the Plan

1. **Navigation Hub overlap** — Today's commit (618aa61) added a "Navigation Hub and dev server." Is this the same effort, or a parallel one?
2. **5 new intelligence screens** — Today's commit (6b3a0fd) added Athena, Impact Sim, Replay, Spatial, and a new Command Center variant. Not in the plan's inventory.
3. **Role-based views** — Commit 2a884b9 added 3-view role system (PM, Site Super, Homeowner). The plan doesn't address role-filtered sidebars.
4. **Badge count API** — The manifest API returns project data, not notification counts. A new endpoint is needed.
5. **Global search** — Procore and Buildertrend both have it. The shell is the right place for it.
6. **Notification system** — No mention of real-time notifications (WebSocket, SSE, or polling).

4.3 Competitive Positioning

> *"If you can provide a software that does it ALL for them, they'll buy it because they all have these problems and they don't work independently."*
>
> — **Scott Rossi**, *Owner, Bespoke Home Builders*

COMPETITOR	PRICE	UNIFIED SHELL?	AI INTELLIGENCE?	BUILDERWRX ADVANTAGE
Buildertrend	$339-$829/mo	Yes	Client comms only	AI scope/estimating, event-sourced financials
Procore	$$$	Yes	Agent Studio (2026)	10-50x cheaper, residential-focused
BuildBook	$79+/mo	Yes	None	Deeper financials, scope management
Buildxact	$169-$199/mo	Partial	Estimating only ("Blu")	Full lifecycle, not just estimating

The iframe shell is invisible to the end user. Scott doesn't care whether screens load via iframes or module federation. He cares that he opens one app, clicks "Schedule," sees his Gantt, clicks "Money," sees his bud-

get. That's what this plan delivers.

5. Recommendations

5.1 Phase 1 Additions (Day 1-2, Not Deferred)

1. **Browser history management** — Push sidebar navigation to `window.history`. Handle `popstate`. Users expect Back to work.
2. **Embedded mode** — `command-center-live.html` and `selection-tracker-live.html` need `?embedded=true` to hide internal sidebars.
3. **"Open in new tab" button** — Small icon in shell top bar. Enables printing and full-width viewing.

5.2 Phase 2 Refinements

4. **Two-way postMessage contract** — Typed message schema for project changes, theme toggles, navigation requests.
5. **iframe detection** — Add `if (window.parent !== window) bar.style.display = 'none'` to `project-picker.js`. One line, affects 12 screens.
6. **Badge count API** — `GET /api/badges/{projectId}` returning `{ inbox: 8, fieldActivity: 3, changeOrders: 3 }`.

5.3 Phase 4 — Same-Origin Deployment

7. **Move Isaac's app behind nginx** — Serve at `ssda.whitelanewrx.com/mc/` via reverse proxy. Eliminates ALL cross-origin issues in one move.

5.4 Phase 5 — Component Migration

8. **Don't rush this.** When the team is ready: extract shared design tokens (`@wrx/tokens`), convert high-traffic screens to React, keep low-traffic screens as iframed HTML. Consider Web Components (Stencil) for framework-agnostic shared elements.

5.5 Strategic Additions

9. **Role-based sidebar** — Config object defining which items each role (PM, Site Super, Homeowner, Owner/GC) sees.
10. **Global search** — Text search that opens the right screen with a filter parameter.
11. **Intelligence screen placement** — Athena, Impact Sim, Replay, Spatial need sidebar slots.

6. Open Questions — Our Position

QUESTION	OUR ANSWER	EVIDENCE
Q1: Can iframes point to Vercel?	YES — today	No `X-Frame-Options` header. `access-control-allow-origin: *`. Tested live.
Q2: Does dark theme override Isaac's CSS?	NO	Hardcoded Tailwind classes, no CSS variables, no postMessage listener.
Q3: Auth — cookie inheritance?	YES, but same-origin only	Cross-origin cookies blocked. Deploy behind nginx first (Rec #7).
Q4: Mobile — Phase 1?	DEFER to Phase 3	Field users use Kestrel mobile. PM uses desktop for Command Center.

7. Data Flow Validation

The plan's data flow diagram is accurate. All screens call Peregrine API endpoints. The shell is "navigation wrapper only" — it doesn't process data.

```
Supabase (50+ tables)
         |
    Peregrine (200+ API endpoints)
         |
    +----+----+----------+----------+
    |         |          |          |
Isaac's App  Argos's    Jess's
(Vercel)    Screens    Screens
Next.js    (vanilla)   (vanilla)
    |         |          |
    +----+----+----------+
         |
   Unified Shell
   (iframe loader)
```

Note: The plan says Peregrine has "200+ endpoints." Our codebase analysis identified 31+ route families. "200+" likely includes parameterized routes counted individually — not wrong, but an aggressive count.

8. Build Sequence — Adjusted Timeline

PHASE	PLAN SAYS	OUR ASSESSMENT	ADJUSTED
Phase 1	Day 1: Shell + wire screens	Add history mgmt + embedded mode	**Day 1-2**
Phase 2	Day 2: Hide picker, wire badges	iframe detection = 1 line. Badge API needs building.	**Day 2-3**
Phase 3	Week 1: Build 4 placeholder screens	Realistic if keeping simple (data tables)	**Week 1**
Phase 4	Week 2: Isaac integration + theme	Cross-origin theme won't work. Use nginx proxy.	**Week 2-3**
Phase 5	Ongoing: Component migration	No rush. iframes work at current scale.	**Month 2+**

9. Conclusion

The Unified Command Center Plan is a well-structured, pragmatic approach to consolidating BuilderWrx's three UI streams. The iframe shell architecture is the right choice for the team's size, tech stack diversity, and speed requirements.

Key corrections needed:

- Not all screens support the shared theme system (`command-center-live.html` is independent)
- iframe detection code doesn't exist yet (correctly identified as Phase 2 but language is misleading)
- 5 new intelligence screens from today's commits need to be added to the inventory
- The Navigation Hub commit may overlap — needs alignment

Key additions recommended:

- Browser history management (Phase 1)
- Embedded mode for dual-sidebar screens (Phase 1)
- Same-origin nginx deployment (Phase 4, replaces cross-origin hacks)
- Role-based sidebar configuration
- Badge count API endpoint

BOTTOM LINE

Ship it. The iframe approach gets a unified product in front of Scott in days, not weeks. The user experience — one sidebar, one app, everything in one place — is what matters for the Keiper demo and beyond.

Sources

- Martin Fowler — Micro Frontends
- Micro-Frontends: Complete Architecture Guide for 2026
- How Microfrontends Work: From iframes to Module Federation
- Microfrontend Implementations Compared
- Micro-Frontends: Are They Still Worth It in 2025?
- MDN — postMessage() Security
- MDN — Same-Origin Policy
- Vercel — How to Enable CORS
- Procore Platform Architecture
- Procore AI Agents Announcement
- Next.js Multi-Zones Documentation

Analysis conducted using: codebase verification (16 HTML screens audited), HTTP header testing (Vercel + SSDA), industry research (micro-frontend patterns, Procore architecture), competitive intelligence, and BuilderWrx product knowledge base.

Prepared by Jessica Mitchell with Philo Workshop Team

White Lane AI — April 8, 2026